



Default JUnit Test Instance Lifecycle

Understanding how JUnit manages test instances and why it matters for writing reliable unit tests.

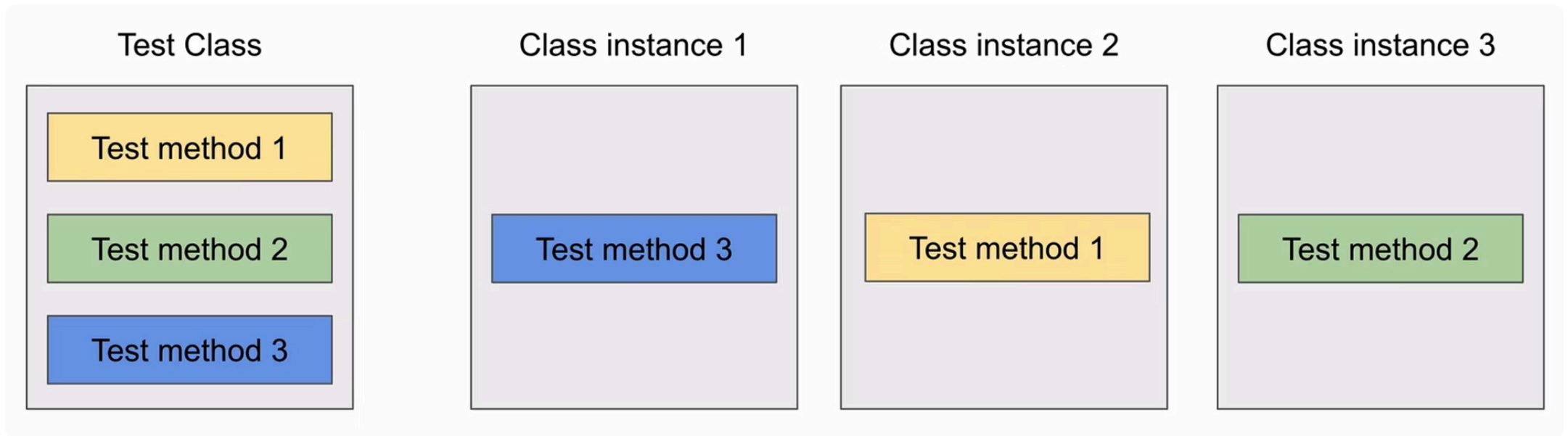


by **Rodrigo Martins Pagliares**

Last update: 12/25/2025

Previously on this course...

- By default, JUnit creates a **new test class instance per test method** (*per-method* lifecycle).
- If a **test class** has three test methods, **three instances** are created.
- Each **test method** runs in complete isolation from the others.
- Instance variables modified in one test are **not visible** in another.



Why Default Isolation Is Important



Prevents Dependencies

Prevents unintended dependencies between test methods.



Any Order Execution

Ensures tests can be executed in **any order** safely.



Deterministic Results

Makes unit tests deterministic and easier to maintain.



Behavior Focus

Encourages testing behavior, not shared state.



Changing the Default Behavior



01

JUnit Flexibility

JUnit allows changing how test instances are created.

02

@TestInstance Annotation

This is done using the `@TestInstance` annotation.

03

One Instance Per Class

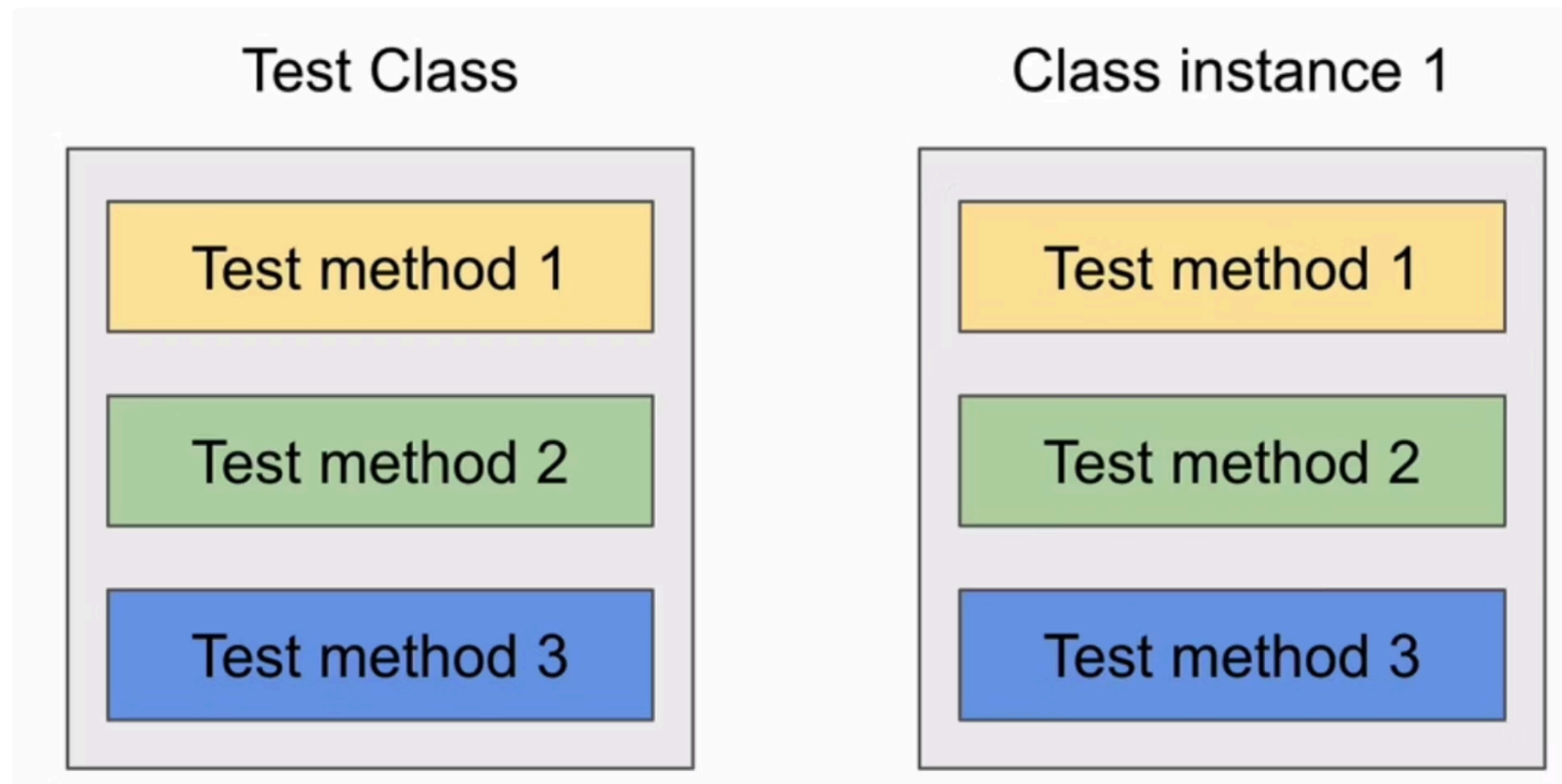
Instead of one instance per method, you can use **one instance per class**.

04

Class-Wide Configuration

This configuration applies to all test methods in the class.

TestInstance Lifecycle: **PER_CLASS**



Single Instance

With **PER_CLASS**, only one test instance is created.

Same Object

All test methods run on the **same object instance**.

Shared Variables

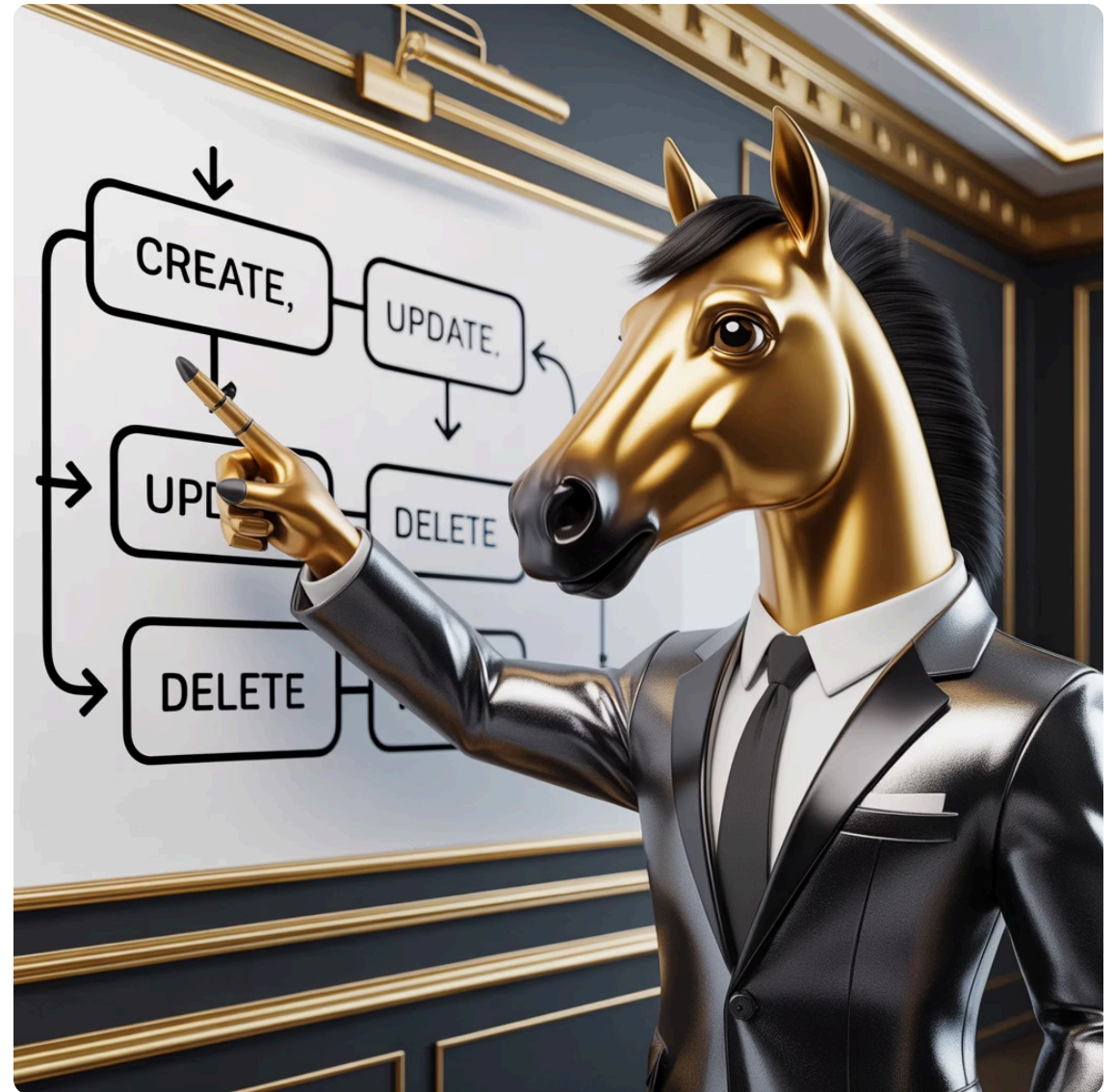
Instance variables can be shared across test methods.

State Exchange

Useful when test methods need to exchange state.

Why PER_CLASS Is Useful for Integration Tests

- Integration tests often follow a logical sequence.
- **Example:** create data → update data → delete data.
- Sharing instance state simplifies this workflow.
- Combined with ordered execution, tests gain more control.



Combining Test Instance and Test Order



Predictable Execution

Ordered tests can run **one after another** predictably.



Shared Variables

A single test instance allows shared instance variables.



Dependent Scenarios

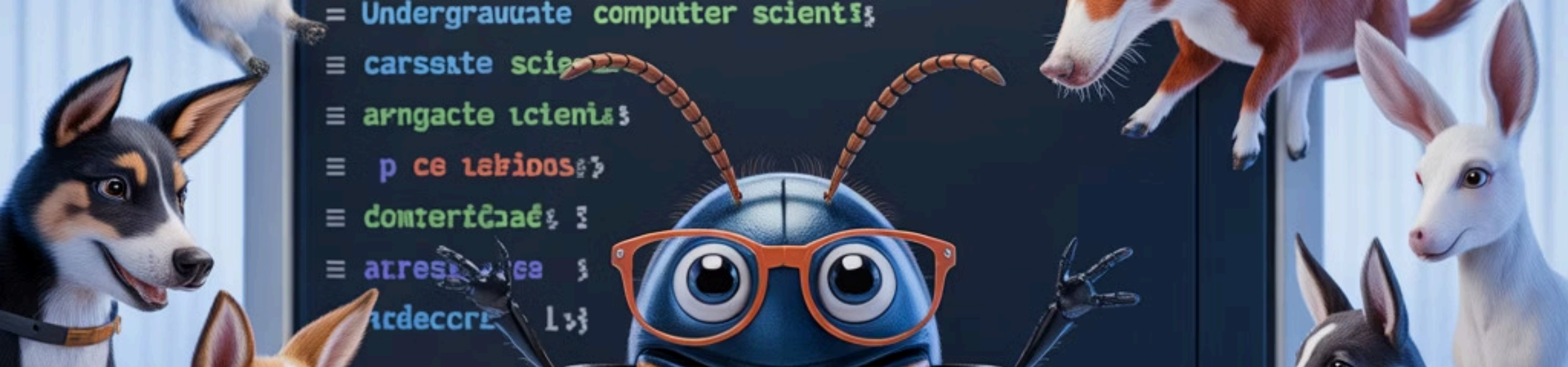
This enables dependent test scenarios when needed.



Use Carefully

Should be used carefully to avoid fragile test designs.





Conclusion

Default Behavior

Default JUnit behavior favors isolation and independence.

Shared State Option

`@TestInstance(PER_CLASS)` allows shared state across tests.

Integration Testing

This is especially useful for integration-style testing.

Increased Control

Combining shared instances with ordered tests increases control.

Open Floor Discussion

- When does shared state improve tests instead of harming them?
- How can PER_CLASS lead to fragile or order-dependent tests?
- Should integration tests be separated from unit tests when using shared instances?

